

Formal Grammars: The Hidden Mathematics of Language



[James Wilkins](#)

Follow

7 min read

.

Apr 22, 2025

Listen

Share

More

Zoom image will be displayed

LANGUAGES AND GRAMMARS

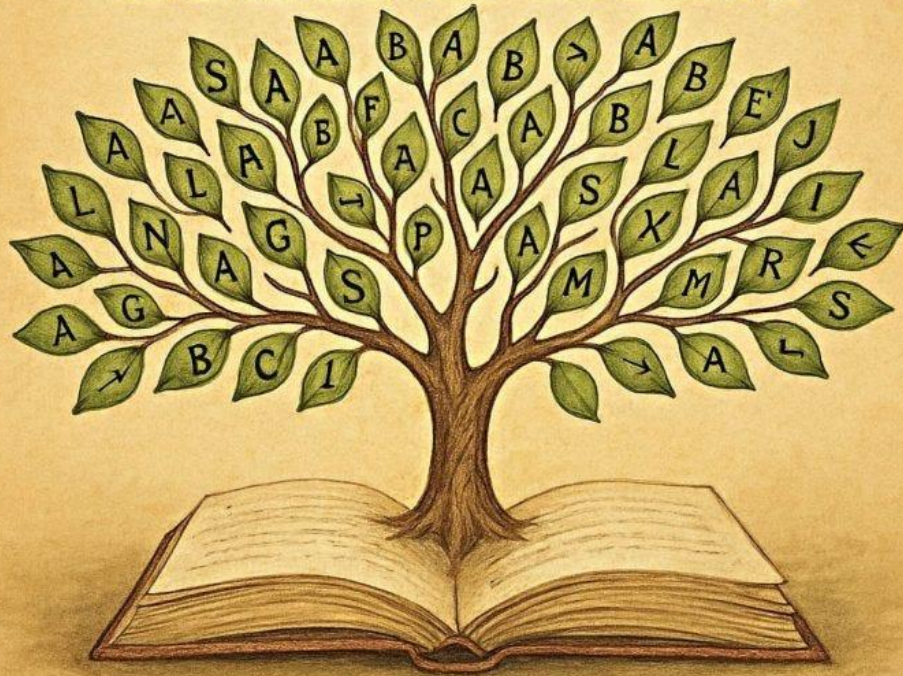


Image generated using DALL-E, OpenAI

*Not a member? You can read this article, **completely free**, at the link below:*

<https://jdhwilkins.com/formal-grammars-the-hidden-mathematics-of-language/>

Invisible Rules of Language

When we speak, we rarely stop to think about the way our words are structured. But behind the scenes, there's a complex set of rules that

we've never been explicitly taught, yet we all internalised from a young age.

Take a simple sentence like:

"The dog chased the cat."

Swap the order of a few words — *"Chased the the dog cat"* — and it stops making sense. Instinctively, we know that English sentences follow a subject-verb-object pattern, which has become so ingrained in us that even a slight disruption makes a sentence feel broken.

Let's see an example of a more complex pattern...

"The red big balloon."

There's nothing *technically* wrong with the sentence, but I'm sure we can all agree (at least if you're a native English speaker) that it just doesn't sound quite right.

Turns out there's a rule that we all subconsciously follow for the order that we place adjectives in a sentence.

Opinion → Size → Age → Shape → Colour → Origin → Material

Most people have never learned this rule explicitly; it's absorbed through exposure, yet it's remarkably consistent across the population. You can pile on as many adjectives as you like too — a lovely little old round green French silver pocket watch — and native speakers will still get the order

right without even thinking about it. Again, the moment the rule is broken, something feels off.

Clearly, the rules governing our language are so complex and nuanced that we don't even know that we know them. It makes you wonder what other patterns we've picked up on that we aren't aware of.

This is where formal grammars come in. They provide a framework for describing the rules of a language in precise, mathematical terms. And as we'll see, they're not just a theoretical concept; they form the foundation of programming language compilers, have applications for NLP and generative AI, and are even used for bioinformatics research.

Languages and Grammars

Let's first take a look at some other definitions we'll need to help us understand what we mean by a **grammar**. For now we'll stay within the context of natural languages (in this case English) but this can be applied to other types of language too (programming languages, formal mathematics, etc...).

Terminals are the words (and potentially symbols) in our language. The set of all terminals in a language is known as an **alphabet**, and sometimes referred to using the symbol Σ .

```
'cat', 'dog', 'ran', 'the', 'because', 'to', '.', 'quickly'
```

Non-terminals are the types of **word**, **phrase**, **clause**, and **sentence** structures that we allow in our language.

VERB, ADJECTIVE, NOUN-PHRASE, SUBORDINATE-**CLAUSE**, SENTENCE

Production rules govern how we can put these non-terminal components together. They tell us which components make up a larger structure.

NOUN-PHRASE --> ADJECTIVE NOUN

We also have replacement rules that map word types to words (non-terminals to terminals). A pipe, “|”, is used to separate different possible options for terminals. The following rule can be read as: “NOUN can be replaced by ‘cat’ or ‘dog’ “.

NOUN --> **cat** | dog

A **string** is a finite sequence of terminals. A **language** is the set of all possible strings that comply with the grammar rules. (Think of a book containing every possible sentence written in English).

And finally, the one we’ve been waiting for...

A **grammar** is a system that describes the strings that are allowed in a language. It consists of 4 parts:

Grammar, $G = (N, \Sigma, P, S)$

- Set of non-terminals, **N**

- Alphabet / set of all terminals,
- Set of all production rules, **P**
- Sentence start symbol, **S** (we'll talk about this one more in a moment, but every grammar must have one)

Simple Grammar Example

While we can't feasibly model the entire English language with a formal grammar, we can model a subset of it. Let's see an example to understand how grammars work in practice.

We'll start with a simple grammar, $G = (N, \Sigma, P, S)$ for basic subject-verb-object sentences.

- $N := \{ S, NP, VP, DET, NOUN, Verb \}$
- $\Sigma := \{ \text{"the"}, \text{"a"}, \text{"cat"}, \text{"dog"}, \text{"fish"}, \text{"eats"}, \text{"chases"}, \text{"likes"} \}$
- $P := \{$
 1. $S \rightarrow NP VP,$
 2. $NP \rightarrow DET NOUN,$
 3. $VP \rightarrow VERB NP,$
 4. $DET \rightarrow \text{"the"} \mid \text{"a"},$
 5. $NOUN \rightarrow \text{"cat"} \mid \text{"dog"} \mid \text{"fish"},$
 6. $VERB \rightarrow \text{"eats"} \mid \text{"chases"} \mid \text{"likes"} \}$

- Sentence start symbol, $S \in N$

Parsing Vs Deriving

There are 2 main ways we can use a grammar: parsing and deriving.

Parsing a string means we are checking if it is valid within the language.

We look through our sentence to find an element that appears on the right-hand side of one of our production rules, then replace that terminal with whatever is on the left-hand side of the rule. We repeat this process, replacing elements (or groups of elements) until we are left with just the sentence start symbol.

```
# sentence we wish to parse:
>>> the dog likes a cat

# Apply rule: DET --> 'the'
>>> DET dog likes a cat

# Apply rule: NOUN --> 'dog'
>>> DET NOUN likes a cat

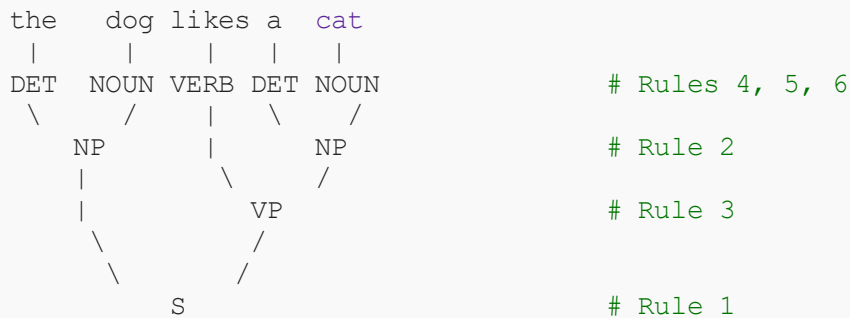
# Apply rule: NP --> DET NOUN
>>> NP likes a cat

...

>>> NP VP

# Apply rule: S --> NP VP
>>> S
```

Sometimes, you'll see these steps drawn as a tree diagram (a parse tree), as it can be easier to see how the rules are applied.



We can tell if our sentence is valid if we can use the production rules to get back to the sentence start symbol, S.

Deriving a string is the same process, but in reverse. We start with the sentence start symbol, S, and use production rules to generate a sentence. You can think of this as the process that we all do subconsciously when we talk or write.

Current Sentinel Form	Rules used:
S	sentence start symbol
NP VP	(S --> NP VP)
(DET NOUN) VP	(NP --> DET NOUN)
(the) NOUN VP	(DET --> 'the')
the (cat) VP	(NOUN --> 'cat')
the cat (VERB NP)	(VP --> VERB NP)
the cat (chases) NP	(VERB --> 'chases')
the cat chases (DET NOUN)	(NP --> DET NOUN)
the cat chases the NOUN	(DET --> 'the')
The cat chases the fish	(NOUN --> 'fish')

(N.B. A **sentinel form** is any mix of terminals and non-terminals formed mid way through parsing or deriving a string. It is not a valid sentence in the language by itself as it contains non-terminals.)

It might seem like we're just shuffling symbols around for the fun of it, but this process of building and checking sentences against a set of rules is exactly how machines process language.

When a compiler checks if your code is valid, it's parsing it using a formal grammar. When an LLM generates a coherent output in a specific format, it's deriving using grammar rules. What we're doing here by hand is the process that computers follow millions of times a second.

Regular Expressions

As we discussed earlier, a language is the set of all strings that satisfy our grammar rules, but how do we actually describe this set of possible sentences? One such tool we can use is Regular Expressions (sometimes known as Regex).

A regular expression is just a concise way of representing a language, especially useful when listing all of the possible sentences isn't feasible.

Regex Notation		Meaning
[A-Z]		Any uppercase letter
[a-z]		Any lowercase letter
[0-9]		Any digit
+		One or more repetitions
*		Zero or more repetitions
?		Zero or one repetition
()		Used to group terms
\b		Word boundary
		separates different options / "or"

Example 1. Simple Sentence Parsing

In this example, we'll use regex to parse some sentences to check if they are valid in the language. Using Regex notation, we can describe our language:

```
(cat|dog) (eats|chases|likes) (fish|cheese)
```

This language is tiny; there's only 12 valid sentences

- “cat eats fish”,
- “dog likes cheese”,
- “cat chases cheese”,
- ...

Let's write a short Python script to implement this. Python comes pre installed with the 're' library, specifically designed for pattern matching using regular expressions.

```
import re
pattern = r"(cat|dog) (eats|chases|likes) (fish|cheese)"
if re.fullmatch(pattern, "dog chases fish"):
    print("Valid sentence!")
else:
    print("Not valid.")
```

```
>>> Valid sentence!
```

Which tells us that “dog chases fish” is a valid sentence in the language.

Example 2. Finding Full Names in Text

```
import re
text = "Alice Johnson met with John Smith and Dr. Emily Stone. Later, they spoke with michael brown and Sarah O'Connor."
pattern = r"\b[A-Z][a-z]+ [A-Z][a-z]+\b"
matches = re.findall(pattern, text)
print(matches)
```

This pattern describes a language in the following format:

word boundary, capital letter, one or more lower case letters, space, capital letter, one or more lower case letter, word boundary.

```
>>> ['Alice Johnson', 'John Smith', 'Emily Stone']
```

Even though it wasn't able to find *all* of the names in the text, it was able to find all of the ones in the given format. Techniques like this can be used for text data processing or web scraping to find and match text when we don't know exactly what we're looking for.

Regular expressions are just another way to represent a language, however, they can't be used to describe **every** type of language. There are 4 main types of grammar defined by the types of production rules they include. Regular expressions can only be used to represent the simplest type of grammar known as a **Regular Grammar**.

Language may feel like second nature to us, but as we've seen, it's underpinned by invisible rules — rules we follow effortlessly but rarely think about. Formal grammars shed lights on the structure of these languages, providing us the tools necessary to analyse and describe the mechanisms at play behind the scenes. We've only just scratched the surface of what's possible!

Language

Mathematics

NLP